INTERNAL ENGINEERING REFERENCE

Impact Reports Developer Documentation

Architecture overview, services, data model, and deployment for the Impact Reports platform — a full-stack Node/React application for publishing field-project impact reports.

Version 1.0

Prepared September 2026

Audience: Engineers onboarding to this codebase

1. Overview & Tech Stack

Impact Reports is a full-stack application for creating, reviewing, and publishing field-project "impact reports" — structured case studies covering a project's background, metrics, media, and locations. It has two audiences: an **admin dashboard** where staff draft and approve reports through a five-step wizard, and a **public portal** where anyone can browse published reports, search/filter them, and view an interactive locations map.

Stack

Layer	Technology
Frontend	React 18 + React Router v7, built with Vite, styled with Tailwind CSS, animated with Framer Motion
Backend	Node.js (ES modules) + Express 4
Database	MongoDB via Mongoose 8 (self-hosted in production; may be MongoDB Atlas in local dev)
Auth	JWT (jsonwebtoken) + bcryptjs; optional email-OTP or TOTP two-factor login
Media storage	Cloudinary (image uploads, 2 MB limit, JPEG/PNG/WebP/GIF)
Email	Nodemailer (SMTP) — invites, OTPs, weekly digest; silently skipped if SMTP isn't configured
Maps	React Leaflet
Charts	ECharts (via echarts-for-react)
Rich text	Tiptap v3 (narrative & milestone fields)
Scheduled jobs	node-cron — weekly digest email, monthly log archival
Process manager	PM2 (production), app name `impact-reports`

> **Module system:** the entire codebase is ES modules (`"type": "module"`). Server code uses `import` / `export` exclusively — never `require()` . Since Node doesn't provide `__dirname` / `__filename` natively in ESM, the codebase uses the `fileURLToPath(import.meta.url)` pattern everywhere it's needed (canonical example: `server/index.js`). The one exception is `ecosystem.config.cjs` , which PM2 requires to be CommonJS.

System Architecture

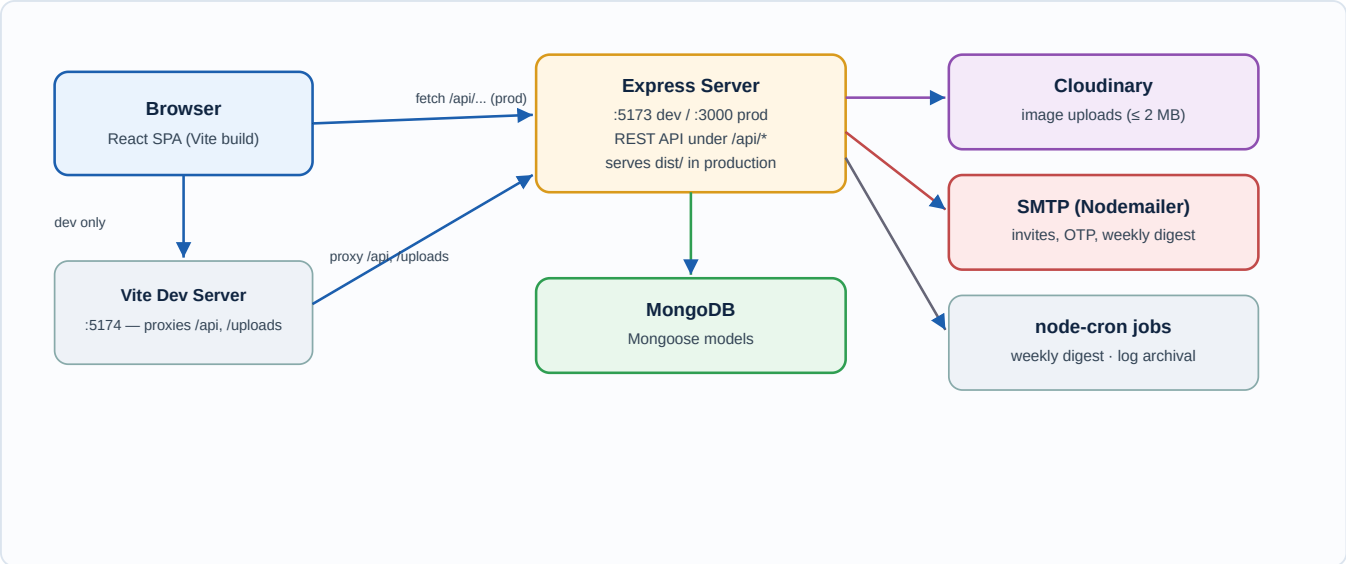


Figure 1 — Runtime architecture. In development, Vite (5174) proxies /api and /uploads to Express (5173). In production a single Express process on :3000 serves the built SPA and the API.

> **Critical dev-startup order:** always start the backend (`npm run server:dev`) before the frontend (`npm run dev`). Vite defaults to port 5173 and only falls back to 5174 if 5173 is taken. If Vite claims 5173 first, the proxy target breaks and API calls fail silently.

2. Repository Layout

```
impact-reports/
├── server/                  Express API
│   ├── index.js             app entrypoint — mounts routes, serves dist/ in prod
│   ├── bootstrap.js         loads .env from project root (must import first)
│   ├── config/db.js         Mongoose connection
│   ├── middleware/auth.js   protect() — verifies JWT, attaches req.admin
│   ├── models/              Mongoose schemas (Report, Admin, Invite, ...)
│   ├── routes/              one file per API namespace (see §4)
│   ├── cron/                weeklyDigest.js, logArchival.js
│   ├── utils/               regionUtils, slugUtils, pagination, ...
│   └── scripts/             seed.js, migrateDemographics.js (one-off/manual)
├── src/                     React app
│   ├── pages/web/           public site: portal, report-detail, locations-map
│   ├── pages/admin/         /alkabirimpact/* admin dashboard + auth pages
│   ├── common/              shared UI: Header, Footer, RichTextEditor, SafeIcon
│   ├── context/AdminAuthContext.jsx   login/logout/getToken, session state
│   ├── services/            adminApi.js (authed fetch), publicApi.js
│   ├── hooks/               usePublicReports, usePortalFilters, ...
│   └── utils/               cookies, location, videoUtils
├── docs/api/                per-route API reference (auth.md, reports.md, ...)
├── vite.config.js           frontend build + dev proxy config
├── vite.server.config.js    SSR-bundles server/ → dist/server/ (build:server)
└── ecosystem.config.cjs     PM2 process config (production)
```

Path aliases

Alias	Resolves to	Used in
`@`	`./src`	`vite.config.js` — the whole frontend
`@server`	`./server`	`vite.server.config.js` — the server SSR bundle only

3. Authentication & Session Model

Admin auth is JWT Bearer-token based. `server/middleware/auth.js` exports `protect`, which verifies the token, loads the `Admin` document, and attaches it as `req.admin` on every protected route. There are three roles: `superadmin` `editor` `viewer` .

Login flow (multi-step)

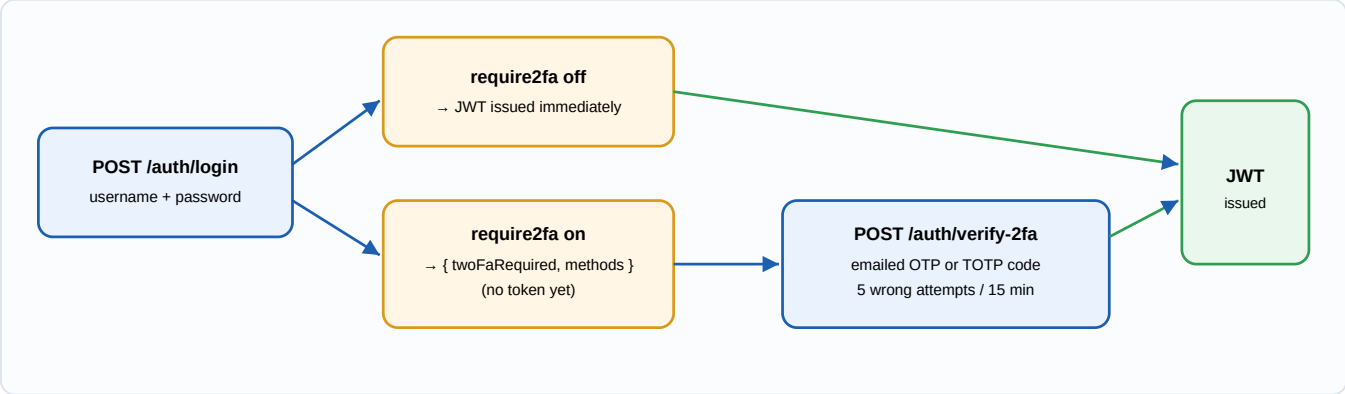


Figure 2 — Login flow. `POST /auth/resend-2fa-otp` (not pictured) re-sends the email OTP during step 2.

1. **POST /api/auth/login** — validates credentials. If the platform's `require2fa` setting is on, returns `{ twoFaRequired: true, methods }` instead of a token.
2. **POST /api/auth/verify-2fa** — verifies the emailed OTP or a TOTP code (selected via a `method` field in the request body); returns the JWT. An in-memory rate limiter allows 5 wrong attempts per 15-minute window (resets on server restart).
3. **POST /api/auth/resend-2fa-otp** — re-sends the email OTP for step 2.

The JWT is stored client-side in an `adminToken` cookie (1-day expiry), with the admin profile mirrored to `localStorage`. `AdminAuthContext` exposes `login`, `logout`, and `getToken`. Any 401 response from `adminApi.js` dispatches a `CustomEvent('admin:unauthorized')`, which clears the session — the same path used by the inactivity timeout.

Inactivity timeout: a watcher polls every 30 seconds, comparing `Date.now()` against an `adminLastActivity` timestamp in `localStorage` (refreshed on mouse/keyboard/touch/scroll/click). The timeout value comes from `PlatformSettings.sessionTimeoutMinutes` (default 30), delivered in the login response.

Password reset flow

`POST /forgot-password` (emails a 6-digit OTP, 10-min TTL) → `POST /verify-reset-otp` (validates OTP, returns a 15-minute `resetToken` carrying a `pendingReset: true` claim) → `POST /reset-password` (consumes the reset token, sets the new password).

4. API Surface

All routes are namespaced under `/api`. Detailed request/response shapes live in `docs/api/*.md` — this is a map of what's where.

Namespace	Auth	Purpose
`/auth`	mixed	Login/2FA/logout, `GET /me`, activity logs (`GET /logs`, CSV/Excel export, superadmin purge), password reset flow
`/admin`	protected	Dashboard KPIs, admin-user CRUD, role/active toggles, own profile/password settings, TOTP setup, slug backfill (superadmin)
`/invite`	public + token	Email-based invite flow; tokens SHA-256-hashed at rest, 48 h expiry
`/reports`	protected	Report CRUD, per-step wizard saves, field-visibility, approve, counts, export
`/upload`	protected	Cloudinary image upload (≤ 2 MB, JPEG/PNG/WebP/GIF)
`/public`	none	Published reports, single report by id/slug, stats, filters, search, locations map
`/config`	mixed	Taxonomy reads (public) / writes (superadmin); platform settings; weekly-digest manual trigger
`/subscribers`	mixed	Public email capture; superadmin list/delete
`/notifications`	protected	Last 30 + unread count; mark read; currently only populated by failed-login events
`/health`	none	Liveness check (defined inline in `server/index.js`)

> **Route-ordering gotcha:** `GET /reports/export` is registered *before* `GET /reports/:id` — otherwise Express would treat the literal string `export` as a MongoDB ObjectId path param.

5. Data Model

Report (core document)

Fields are grouped by which wizard step owns them (`STEP_FIELDS` in `server/routes/reports.js`); each step's `PATCH /reports/:id/step/:stepName` only touches its own field list.

Step	Fields
general	`projectName, assignedTo, approvalAssignedTo, actualProjectId, shortSubtitle, cost, programLead, implementingPartner, partnerOrganizations, reportingPeriod, region, sector, livesImpacted`
narrative	`projectBackground, strategicResponse, testimonialsTitle, testimonials[]`
metrics	`demographicsTitle, demographics[], deliverables[]`
media	`images[], thumbnail, videoUrl, milestones[]`
location	`countries[], locations[], donationUrl, ctaTitle, ctaDescription, ctaButtonText`

Cross-cutting fields: `slug` , `projectId` (sequential, via `Counter.next()`), `status` , `stepsCompleted[]` , `hiddenFields[]` , `createdBy` .

Report lifecycle

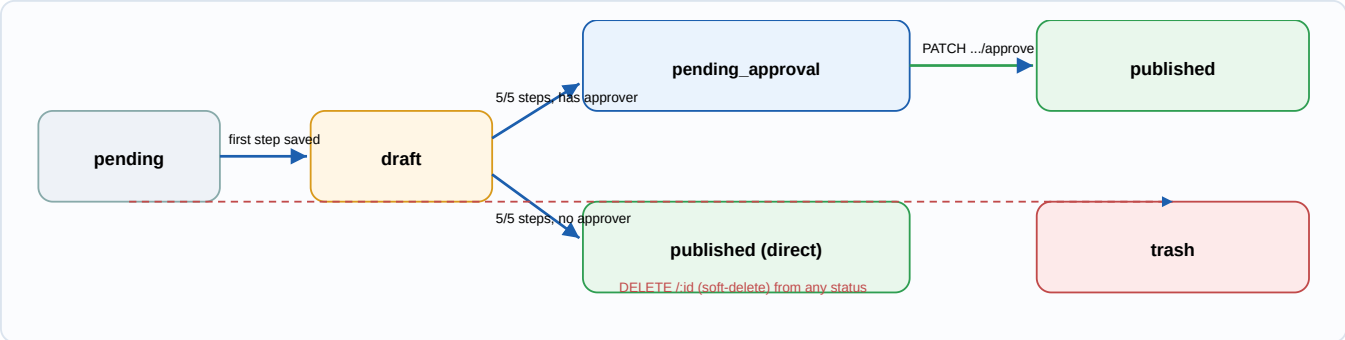


Figure 3 — Report status transitions. Trash supports restore (`PATCH /:id/restore`) or permanent delete (`DELETE /:id/permanent`).

A report is created with `status: 'pending'` and moves to `'draft'` as wizard steps are saved. Once all five steps (`general, narrative, metrics, media, location`) appear in `stepsCompleted` : if `approvalAssignedTo` is non-empty, status becomes `'pending_approval'` (awaiting sign-off via `PATCH /reports/:id/approve`); otherwise it auto-publishes straight to `'published'` . Only published reports appear on public endpoints.

Access control on a report

Helper	Grants access when…
`canEdit(admin, report)`	admin is `superadmin` , or admin's id is in `report.assignedTo`
`canView(admin, report)`	`canEdit` is true, or admin's id is in `report.approvalAssignedTo`

Other models

Model	Purpose	Notable fields

Admin	Dashboard user	`username, email, password (bcrypt, 12 rounds), role, region, isActive, failedLoginAttempts, totpSecret/totpEnabled, resetOTPCode/Expires, loginOTPCode/Expires`. `toSafeObject()` strips all secrets before sending to the client.
PlatformSettings	Singleton (`key: 'platform'`)	`defaultRegion, sessionTimeoutMinutes, require2fa, enforceStrongPasswords, weeklyDigest, donationUrl, heroTitle/heroBackgroundImage, regions[]/sectors[]/departments[], socialLinks{}, socialLinksVisible{}`
Counter	Atomic auto-increment	`Counter.next(name)` — used for sequential `Report.projectId`
Subscriber	Newsletter signups	`email` unique; duplicate-key (11000) errors are silently treated as success to avoid email enumeration
Notification	System alerts	`type, title, message, read` — currently only failed-login events populate this
Invite	Pending registrations	`tokenHash` (SHA-256), `expiresAt` (48 h), `role/region/department` pre-set by the inviter, `used`
ActivityLog	Login/logout/purge audit trail	`type, username, role, ip, archivedAt` — monthly cron archives records older than the retention window

> **Freeform taxonomy is deliberate:** `Report.region` is validated only for presence, not membership in `PlatformSettings.regions`. Day-to-day report entry can't produce a stray value because the General-tab dropdown only offers values from `GET /api/config/regions`, but a superadmin editing the taxonomy directly (`PUT /api/config/regions`) can put anything in that list. A hardcoded enum was tried and reverted in favor of full editability — treat this as intentional, not a bug. Related: `server/utils/regionUtils.js` 's `deriveRegion(countries)` is a read-time-only fallback in `public.js` for older reports missing a stored region — it never runs on save and never overwrites a stored value.

6. Frontend Architecture

Routing (React Router v7)

Path	Page
`/`	Public portal — browse published reports
`/report/:slug`	Public report detail
`/locations`	Public "All Locations Map" — country pins + per-country report grid
`/alkabirimpact/login`	Admin credential login (multi-step: login → OTP/TOTP → dashboard)
`/alkabirimpact/register`	Complete registration from an invite token
`/alkabirimpact/dashboard/:section`	Protected admin dashboard, module switched by URL param

Admin dashboard modules

:section	Component	Visible to
`overview`	AdminDashboard	all roles
`reports`	ReportsModule	all roles
`users`	UserManager	superadmin only
`data`	DataManagement	superadmin only
`subscribers`	SubscribersModule	superadmin only
`settings`	Settings	all roles

The dashboard's `page.jsx` defines `VALID_SECTIONS` per role; navigating to a restricted section silently redirects to `overview`. `ReportsModule` tracks the active report and wizard step as URL search params (`?reportId=&step=`) — each "Next" click persists the current step via `PATCH /api/reports/:id/step/:stepName`.

Two-tier map architecture

There are two distinct React Leaflet components, easy to conflate: `LocationsMapCanvas` (locations-map page only) plots one pin per *country* from `/api/public/locations-map`. `LocationsImplementedMapCanvas` plots individual *project sites* with real/derived GPS coordinates, and is imported directly by both the locations-map page and the report-detail page's `LocationsSection` — it is not duplicated. Every `MapContainer` in the app uses `attributionControl={false}` plus an explicit bottom-right `<AttributionControl>`; match this convention for any new map view.

Field visibility

Each report stores `hiddenFields: [String]`. Editors toggle individual fields per-report via the `FieldToggle` component present in every wizard tab; toggles save to `PATCH /api/reports/:id/field-visibility`. The public report-detail page converts the array to a `Set` and passes it as a `hiddenFields` prop to every display component, each of which conditionally omits the corresponding content.

Key services & utilities

File	Role
`services/adminApi.js`	Authenticated fetch wrapper — reads the `adminToken` cookie, dispatches `admin:unauthorized` on 401
`services/publicApi.js`	Unauthenticated fetch wrapper for public endpoints
`hooks/usePublicReports`	Fetches paginated reports + stats/filter options
`hooks/usePortalFilters`	URL-param filter/search/page state, 400 ms debounce
`hooks/usePublicLocationsMap`	Fetches `/api/public/locations-map`
`utils/cookies.js`	`getCookie` / `setCookie` / `deleteCookie` for the admin JWT cookie
`utils/location.js`	`cleanCountryName()` ; `getGpsData()` generates deterministic pseudo-coordinates when no real GPS is stored

7. Deployment

Pushing to the `deploy` branch triggers a GitHub Actions workflow that: installs dependencies, runs `npm run lint:error`, runs `npm run build:all` (frontend + server bundle), then SSHs into the production server to `git pull`, `npm ci`, rebuild, and restart via PM2 (`pm2 restart impact-reports --update-env`).

Production runs on **port 3000** (set in `ecosystem.config.cjs`'s `env_production`, overriding the default 5173). `npm run start` sets `NODE_ENV=production`, which makes Express serve the built `dist/` folder and handle the SPA catch-all — without it, `GET /` returns 404.

Production MongoDB is self-hosted (8.0) on the same droplet, bound to `127.0.0.1:27017` with auth enabled — not Atlas. Local development may still point at an Atlas cluster.

Required environment variables

Variable	Purpose
`MONGODB_URI`	MongoDB connection string
`JWT_SECRET`, `JWT_EXPIRES_IN`	JWT signing (e.g. `1d`)
`CLOUDINARY_CLOUD_NAME/API_KEY/API_SECRET`	Image uploads
`CORS_ORIGIN`	Comma-separated allowed origins (default `http://localhost:5174`)
`PORT`	Default `5173`
`APP_URL`	Base URL for invite-email links (default `http://localhost:5174`)
`SMTP_HOST/PORT/SECURE/USER/PASS/FROM`	Nodemailer — emails silently skipped if `SMTP_HOST` is absent (dev fallback)

The backend loads `.env` from the **project root** (not `server/`) via `server/bootstrap.js`, which must be the first import in `server/index.js`.

8. Common Commands

```
# Terminal 1 — backend (start first!)
npm run server:dev      # Express + nodemon on :5173

# Terminal 2 — frontend
npm run dev             # Vite dev server on :5174

npm run lint            # ESLint (also runs before build)
npm run lint:error      # ESLint, errors only
npm run build           # lint + Vite build → dist/
npm run build:server    # SSR-bundles Express → dist/server/
npm run build:all       # build + build:server
npm run seed             # seed MongoDB with initial data
npm run migrate:demographics   # one-time: old men/women/children fields → demographics[]
npm run server           # backend without nodemon
npm run start             # production mode (serves dist/)
npm run preview            # preview the Vite build locally
```

Notable disabled ESLint rules (`eslint.config.js`): `react-hooks/exhaustive-deps` , `no-unused-vars` , `no-case-declarations` , `no-useless-catch` , `react-refresh/only-export-components` are all off. There is no automated test suite in this repository.